

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)**

Department of Computer Science and Engineering

ASSESSMENT TOOL 3– ASSIGNMENT

Course Code:	CSA0901	Course Title:	Programming in Java
CO Assessed:	CO3 – Precision (BL3,BL4)	Assessment:	ASSESSMENT TOOL 3 - ASSIGNMENT
Weightage:		Total Marks:	100
CO1 Statement:	Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques		
Student Name:		Reg. No.:	
Section Batch: /	SLOT B	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 10 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. **Exception Handling** – Write a Java program to demonstrate the use of try, catch, and finally blocks for handling multiple built-in exceptions. **[10 Marks]**
2. **User-Defined Exception** – Write a Java program to create a user-defined InvalidAgeException and check whether a person is eligible to vote. **[10 Marks]**
3. **Multiple Exceptions** – Write a Java program to demonstrate the handling of ArithmeticException, ArrayIndexOutOfBoundsException, and NumberFormatException using appropriate catch blocks. **[10 Marks]**

4. **Creating Threads** – Write a Java program to create two threads using the Thread class. One thread should display numbers from 1 to 10, while the other displays their squares. [10 Marks]
5. **Runnable Interface** – Write a Java program to create multiple threads by implementing the Runnable interface. Display the name and ID of each thread during execution. [10 Marks]
6. **Thread Priority** – Write a Java program to create three threads with different priorities and display their priority values along with their execution details. [10 Marks]
7. **Synchronization** – Write a Java program to demonstrate synchronization by allowing two threads to access and update a shared bank account safely. [10 Marks]
8. **inter-Thread Communication** – Write a Java program to implement a simple producer-consumer problem using wait() and notify(). [10 Marks]
9. **Multiple Threads** – Write a Java program to create three threads that perform three different tasks simultaneously: displaying numbers, calculating the sum of numbers, and displaying even numbers. [10 Marks]
10. **Thread Control** – Write a Java program to demonstrate safe thread suspension, resumption, and termination using a control flag and interrupt() instead of the deprecated suspend(), resume(), and stop() methods. [10 Marks]

1. Exception Handling - try, catch, finally

Program

```
import java.util.Scanner;
class ExceptionDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter numerator: ");
            int a = sc.nextInt();

            System.out.print("Enter denominator: ");
            int b = sc.nextInt();

            System.out.println("Result = " + (a / b));

            int[] arr = {10, 20, 30};
            System.out.print("Enter array index: ");
```

```

        int index = sc.nextInt();

        System.out.println("Element = " + arr[index]);
    }

    catch (ArithmeticException e) {
        System.out.println("Error: Cannot divide by zero");
    }

    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Error: Invalid array index");
    }

    catch (Exception e) {
        System.out.println("Error: Invalid input");
    }

    finally {
        System.out.println("Finally block executed");
    }

    System.out.println("Program completed");
}

```

Input

Enter numerator: 10
Enter denominator: 0

Output

Error: Cannot divide by zero
Finally block executed
Program completed

2. User-Defined Exception - InvalidAgeException

Program

```

import java.util.Scanner;
class InvalidAgeException extends Exception {

    InvalidAgeException(String msg) {
        super(msg);
    }
}
class VoteCheck {

    static void checkAge(int age) throws InvalidAgeException {

        if (age < 18) {
            throw new InvalidAgeException(

```

```

        "Invalid Age: Person is not eligible to vote"
    );
}

System.out.println("Person is eligible to vote");
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter age: ");
    int age = sc.nextInt();

    try {
        checkAge(age);
    }

    catch (InvalidAgeException e) {
        System.out.println(e.getMessage());
    }
}

```

Input

Enter age: 16

Output

Invalid Age: Person is not eligible to vote

3. Multiple Exceptions

Program

```

import java.util.Scanner;
class MultipleExceptionDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter a number: ");
            int a = sc.nextInt();

            System.out.print("Enter divisor: ");
            int b = sc.nextInt();

            System.out.println("Result = " + (a / b));

            int[] arr = {10, 20, 30};

            System.out.print("Enter array index: ");

```

```

        int index = sc.nextInt();

        System.out.println("Element = " + arr[index]);

        System.out.print("Enter a numeric string: ");
        String str = sc.next();

        int n = Integer.parseInt(str);

        System.out.println("Number = " + n);
    }

    catch (ArithmeticException e) {
        System.out.println("ArithmeticException: Cannot divide by
zero");
    }

    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println(
            "ArrayIndexOutOfBoundsException: Invalid index"
        );
    }

    catch (NumberFormatException e) {
        System.out.println(
            "NumberFormatException: Invalid number format"
        );
    }
}

```

Input

```

Enter a number: 10
Enter divisor: 2
Enter array index: 1
Enter a numeric string: abc

```

Output

```

Result = 5
Element = 20
NumberFormatException: Invalid number format

```

4. Creating Threads - Thread Class

Program

```

class NumberThread extends Thread {

    public void run() {

        for (int i = 1; i <= 10; i++) {

```

```

        System.out.println("Number: " + i);
    }
}
class SquareThread extends Thread {

    public void run() {

        for (int i = 1; i <= 10; i++) {
            System.out.println("Square of " + i + " = " + (i * i));
        }
    }
}
class ThreadDemo {

    public static void main(String[] args) {

        NumberThread t1 = new NumberThread();
        SquareThread t2 = new SquareThread();

        t1.start();
        t2.start();
    }
}

```

Input

No input required.

Output

The order may vary because both threads execute concurrently.

```

Number: 1
Square of 1 = 1
Number: 2
Square of 2 = 4
Number: 3
Square of 3 = 9
Number: 4
Square of 4 = 16
...
Number: 10
Square of 10 = 100

```

5. Runnable Interface

Program

```

class MyRunnable implements Runnable {

    public void run() {

        Thread t = Thread.currentThread();

        System.out.println(
            "Thread Name: " + t.getName() +
            ", Thread ID: " + t.getId()
        );
    }
}

```

```

        );
    }
}
class RunnableDemo {

    public static void main(String[] args) {

        MyRunnable obj = new MyRunnable();

        Thread t1 = new Thread(obj, "Thread-1");
        Thread t2 = new Thread(obj, "Thread-2");
        Thread t3 = new Thread(obj, "Thread-3");

        t1.start();
        t2.start();
        t3.start();
    }
}

```

Input

No input required.

Output

```

Thread Name: Thread-1, Thread ID: 12
Thread Name: Thread-2, Thread ID: 13
Thread Name: Thread-3, Thread ID: 14

```

Note: Thread IDs may be different on different executions.

6. Thread Priority

Program

```

class PriorityThread extends Thread {

    PriorityThread(String name) {
        super(name);
    }

    public void run() {

        System.out.println(
            "Thread Name: " + getName()
        );

        System.out.println(
            "Priority: " + getPriority()
        );

        System.out.println(
            "Thread is executing"
        );
    }
}
class PriorityDemo {

```



```

public static void main(String[] args) {

    PriorityThread t1 =
        new PriorityThread("Low Priority Thread");

    PriorityThread t2 =
        new PriorityThread("Normal Priority Thread");

    PriorityThread t3 =
        new PriorityThread("High Priority Thread");

    t1.setPriority(Thread.MIN_PRIORITY);
    t2.setPriority(Thread.NORM_PRIORITY);
    t3.setPriority(Thread.MAX_PRIORITY);

    t1.start();
    t2.start();
    t3.start();

}

```

Input

No input required.

Output

Thread Name: Low Priority Thread

Priority: 1

Thread is executing

Thread Name: Normal Priority Thread

Priority: 5

Thread is executing

Thread Name: High Priority Thread

Priority: 10

Thread is executing

Note: Thread priority affects scheduling preference; it does **not guarantee execution order**.

7. Synchronization - Shared Bank Account

Program

```

class BankAccount {

    private int balance = 1000;

    synchronized void deposit(int amount) {

        balance = balance + amount;
    }
}

```

```

        System.out.println(
            Thread.currentThread().getName() +
            " deposited " + amount +
            ", Balance = " + balance
        );
    }

    synchronized void withdraw(int amount) {

        if (amount <= balance) {

            balance = balance - amount;

            System.out.println(
                Thread.currentThread().getName() +
                " withdrew " + amount +
                ", Balance = " + balance
            );
        }

        else {
            System.out.println("Insufficient balance");
        }
    }

    int getBalance() {
        return balance;
    }
}

class BankThread extends Thread {

    BankAccount account;

    BankThread(BankAccount account) {
        this.account = account;
    }

    public void run() {

        account.deposit(500);
        account.withdraw(300);
    }
}

class SynchronizationDemo {

    public static void main(String[] args)
        throws InterruptedException {

        BankAccount account = new BankAccount();

        BankThread t1 = new BankThread(account);

```

```

        BankThread t2 = new BankThread(account);

        t1.setName("Thread-1");
        t2.setName("Thread-2");

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(
            "Final Balance = " + account.getBalance()
        );
    }
}

```

Input

No input required.

Output

```

Thread-1 deposited 500, Balance = 1500
Thread-1 withdrew 300, Balance = 1200
Thread-2 deposited 500, Balance = 1700
Thread-2 withdrew 300, Balance = 1400
Final Balance = 1400

```

Note: The order of the two threads may vary, but synchronization ensures that the balance is updated safely.

8. Inter-Thread Communication - Producer-Consumer

Program

```

class Shared {

    int value;
    boolean available = false;

    synchronized void produce(int value) {

        while (available) {
            try {
                wait();
            }
            catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }

        this.value = value;
    }
}

```

```

        available = true;

        System.out.println("Produced: " + value);

        notify();
    }

    synchronized void consume() {

        while (!available) {
            try {
                wait();
            }
            catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }

        System.out.println("Consumed: " + value);

        available = false;

        notify();
    }
}

class Producer extends Thread {

    Shared s;

    Producer(Shared s) {
        this.s = s;
    }

    public void run() {

        for (int i = 1; i <= 5; i++) {
            s.produce(i * 100);
        }
    }
}

class Consumer extends Thread {

    Shared s;

    Consumer(Shared s) {
        this.s = s;
    }

    public void run() {

```

```

        for (int i = 1; i <= 5; i++) {
            s.consume();
        }
    }
}
class ProducerConsumerDemo {

    public static void main(String[] args)
        throws InterruptedException {

        Shared s = new Shared();

        Producer p = new Producer(s);
        Consumer c = new Consumer(s);

        p.start();
        c.start();

        p.join();
        c.join();

        System.out.println("Process completed");
    }
}

```

Input

No input required.

Output

```

Produced: 100
Consumed: 100
Produced: 200
Consumed: 200
Produced: 300
Consumed: 300
Produced: 400
Consumed: 400
Produced: 500
Consumed: 500
Process completed

```

9. Multiple Threads - Three Different Tasks

Program

```

class NumberThread extends Thread {

    public void run() {

        for (int i = 1; i <= 10; i++) {
            System.out.println("Number: " + i);
        }
    }
}

```

```

    }
}
class SumThread extends Thread {

    public void run() {

        int sum = 0;

        for (int i = 1; i <= 10; i++) {
            sum = sum + i;
        }

        System.out.println("Sum = " + sum);
    }
}
class EvenThread extends Thread {

    public void run() {

        for (int i = 1; i <= 10; i++) {

            if (i % 2 == 0) {
                System.out.println("Even: " + i);
            }
        }
    }
}
class MultipleThreadDemo {

    public static void main(String[] args) {

        NumberThread t1 = new NumberThread();
        SumThread t2 = new SumThread();
        EvenThread t3 = new EvenThread();

        t1.start();
        t2.start();
        t3.start();
    }
}

```

Input

No input required.

Output

```

Number: 1
Number: 2
...
Number: 10
Sum = 55
Even: 2
Even: 4
Even: 6
Even: 8
Even: 10

```

Note: The exact order of output can vary because the three threads execute simultaneously.

10. Thread Control - Suspension, Resumption and Termination

Program

```
class ControlledThread extends Thread {

    private volatile boolean running = true;
    private volatile boolean suspended = false;

    public void run() {

        int i = 1;

        while (running) {

            synchronized (this) {

                while (suspended && running) {

                    try {
                        wait();
                    }

                    catch (InterruptedException e) {
                        Thread.currentThread().interrupt();
                    }
                }

                if (!running) {
                    break;
                }

                System.out.println("Thread running: " + i++);

                try {
                    Thread.sleep(500);
                }

                catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }

            System.out.println("Thread terminated");
        }
    }
}
```

```

    }

    synchronized void suspendThread() {
        suspended = true;
    }

    synchronized void resumeThread() {
        suspended = false;
        notify();
    }

    void stopThread() {
        running = false;
        interrupt();

        synchronized (this) {
            notify();
        }
    }
}

class ThreadControlDemo {

    public static void main(String[] args)
        throws InterruptedException {

        ControlledThread t = new ControlledThread();

        t.start();

        Thread.sleep(1500);

        System.out.println("Suspending thread...");
        t.suspendThread();

        Thread.sleep(1500);

        System.out.println("Resuming thread...");
        t.resumeThread();

        Thread.sleep(1500);

        System.out.println("Terminating thread...");
        t.stopThread();

        t.join();

        System.out.println("Main thread completed");
    }
}

```

Input

No input required.

Output

```
Thread running: 1
Thread running: 2
Thread running: 3
Suspending thread...
Resuming thread...
Thread running: 4
Thread running: 5
Thread running: 6
Terminating thread...
Thread terminated
Main thread completed
```